



**UNIVERSITY OF GHANA**  
College of Basic and Applied Sciences  
School of Engineering Sciences  
Department of Computer Engineering

---

CPEN 208 — Introduction to Software Engineering

# Project 1

A Relational Database and Next.js 14 Application for the  
Computer Engineering Department System (CEDS)

|              |                                   |
|--------------|-----------------------------------|
| Author       | <b>Daniel Botchway</b>            |
| Index number | <b>22381577</b>                   |
| Programme    | <b>BSc Computer Engineering</b>   |
| Level        | <b>200</b>                        |
| Semester     | <b>Second Semester, 2025/2026</b> |
| Database     | <b>PostgreSQL 16.13</b>           |
| Application  | <b>Next.js 14.2 · TypeScript</b>  |

# Table of Contents

---

1. Introduction and Scope
  - 1.1 The problem
  - 1.2 How the brief maps onto the deliverables
2. Database Design
  - 2.1 Why five schemas
  - 2.2 Entity-relationship design
  - 2.3 Table-by-table walkthrough
  - 2.4 Keys, constraints and data integrity
  - 2.5 Indexing strategy
  - 2.6 Triggers and maintained values
3. The Required Function — Outstanding Fees as a JSON Array
  - 3.1 Specification
  - 3.2 Implementation
  - 3.3 Worked example
  - 3.4 Proof of correctness
4. Supporting Stored Functions
5. Sample Data — the CPEN 208 Class
  - 5.1 What was loaded
  - 5.2 Privacy assumption
6. The Next.js 14 Application
  - 6.1 Architecture
  - 6.2 Register, login and sessions
  - 6.3 The dashboard
7. Assumptions Made
8. Testing and Verification
9. Backup and Restore
10. How to Run Everything
11. Challenges, Limitations and Future Work
12. Conclusion

# 1. Introduction and Scope

## 1.1 The problem

The Department of Computer Engineering intends to develop software that keeps five kinds of record: student personal information, student fees payments, course enrollment, the assignment of lecturers to courses, and the assignment of teaching assistants to lecturers. Project 1 asks for the relational database underneath that software, and for a Next.js 14 application with login, registration and a dashboard on top of it.

At present these five things live in different places — a spreadsheet of the class list, the Finance Office's own ledger, a paper registration form, and the Head of Department's teaching-allocation memo. Nothing reconciles them. The question *"how much does this student still owe?"* cannot be answered without opening two systems and trusting that both are current. The database built here exists to make that question answerable with one call.

## 1.2 How the brief maps onto the deliverables

| Requirement in the brief                                                       | Where it is satisfied                                    |
|--------------------------------------------------------------------------------|----------------------------------------------------------|
| Create a Database                                                              | 00_create_database.sql — creates cpen208_ceds            |
| Create Schema(s) when needed                                                   | 01_schemas.sql — 5 schemas, 15 enum types, 3 domains     |
| Create all Tables needed for the functionality                                 | 02_tables.sql — 23 tables                                |
| Create Insert scripts that populate all the tables                             | 06_seed_data.sql — every table populated                 |
| <b>Database function: outstanding fees per student, output as a json array</b> | 04_functions.sql →<br>finance.fn_outstanding_fees_json() |
| Use your class as the data sample                                              | The CPEN 208 Level-200 class, 32 students (§5)           |
| Make all necessary assumptions                                                 | §7, and inline comments throughout the scripts           |
| Use PostgreSQL                                                                 | PostgreSQL 16.13                                         |
| Next.js 14 application with login, register, dashboard                         | web/ — §6                                                |
| Backup of the database                                                         | database/backup/ — 4 formats, restore-tested             |

|         |        |           |       |         |             |
|---------|--------|-----------|-------|---------|-------------|
| 5       | 23     | 23        | 6     | 97      | 73          |
| SCHEMAS | TABLES | FUNCTIONS | VIEWS | INDEXES | CONSTRAINTS |

## 2. Database Design

---

### 2.1 Why five schemas

The brief says to create schemas *when needed*. Twenty-three tables in a single `public` schema would work, but it would force prefix naming ( `fees_payment` , `academic_course` ) to keep related tables together visually, and it would make it impossible to grant an application account access to fees without also granting it access to everything else. The tables are therefore grouped by bounded context:

| Schema                 | Contents                                                                                                     | Rationale                                                                                   |
|------------------------|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <code>core</code>      | department, programme, academic_year, semester                                                               | Reference data every other schema points at                                                 |
| <code>people</code>    | person, student, next_of_kin, lecturer, teaching_assistant                                                   | <b>Functionality 1</b> ; a single source of truth for identity                              |
| <code>academics</code> | course, course_prerequisite, course_offering, enrollment, lecturer_course_assignment, lecturer_ta_assignment | <b>Functionalities 3, 4, 5</b>                                                              |
| <code>finance</code>   | fee_structure, fee_item, student_bill, bill_line, payment                                                    | <b>Functionality 2</b> ; the schema with the tightest access requirements                   |
| <code>app</code>       | app_user, user_session, audit_log                                                                            | Authentication for the web app and the API — deliberately separate from <code>people</code> |

Keeping `app` separate from `people` matters: a login is not a person. An administrator account has no personal record, and a person may exist in the department's records for years before ever creating a login. Conflating them would force nullable password columns onto the student table.

## 2.2 Entity-relationship design

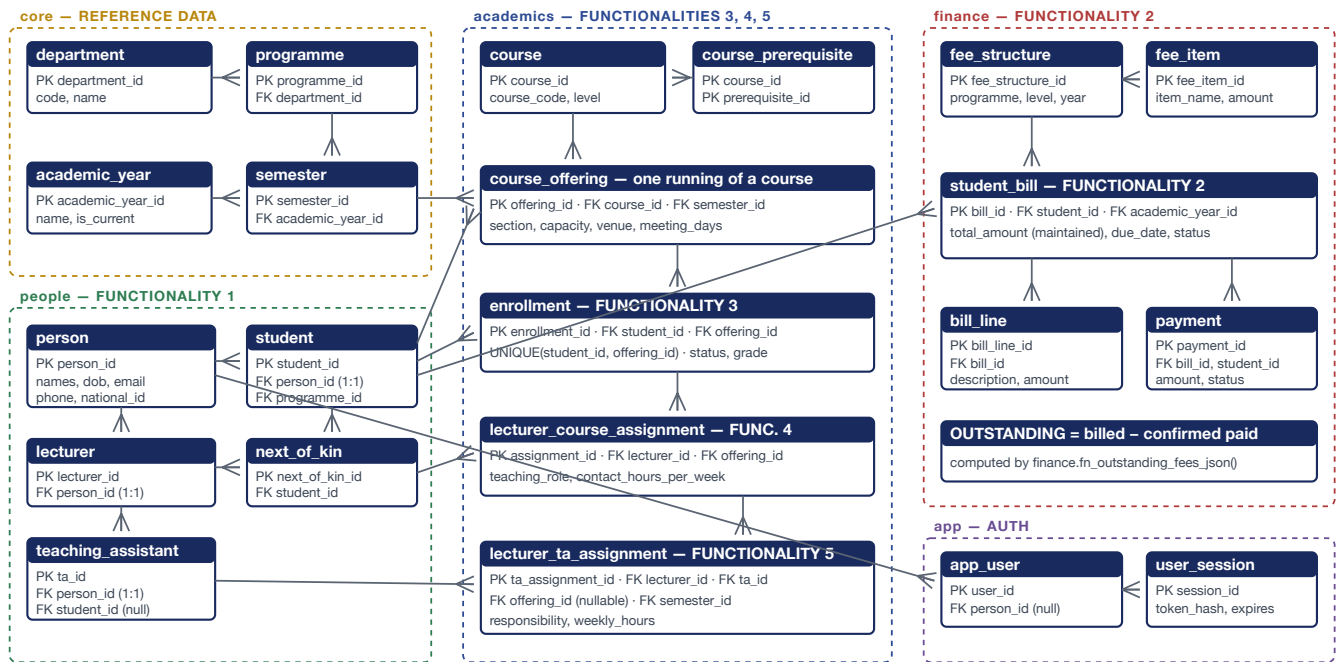


Figure 1 — Entity-relationship design. Crow's-feet point to the "many" side. Grouping boxes show the schema each entity belongs to and the functionality it serves.

### The three design decisions worth defending

**(a) A person supertype.** Students, lecturers and teaching assistants all have a name, a date of birth, an e-mail address and a phone number. Repeating those columns in three tables would mean that a teaching assistant who is also a postgraduate student — which is the normal case — appears twice, with two e-mail addresses that can silently diverge. `people.person` holds the shared attributes once; `student`, `lecturer` and `teaching_assistant` are role tables each holding a `UNIQUE` foreign key to a person plus the columns unique to that role. In the sample data, TA/2025/001 and TA/2025/002 are the same people as MPhil students 24500112 and 24500198 — one person row each.

**(b) course is separate from course\_offering.** CPEN 208 as a catalogue entry — its code, title and credit hours — is stable for years. CPEN 208 as taught in First Semester 2025/2026 has a lecturer, a venue, a capacity and a class list, all of which change every session. Enrolment and teaching assignment therefore reference the *offering*, not the course. Without this split, registering for CPEN 208 in two different years would be indistinguishable, and a lecturer could not be recorded as having taught it in one year but not the next.

**(c) bill\_line copies the fee items rather than referencing them.** When a bill is generated, the amounts from the price list are copied onto the bill. If the department raises the Academic Facility User Fee next year, last year's bills must not silently change. The frozen copy is what makes the outstanding-fees figure historically truthful.

## 2.3 Table-by-table walkthrough

| Table                                | Rows | Purpose                                                           |
|--------------------------------------|------|-------------------------------------------------------------------|
| core.department                      | 3    | Computer Engineering, Mathematics, and the UGRC office            |
| core.programme                       | 2    | BSc and MPhil Computer Engineering                                |
| core.academic_year                   | 2    | 2024/2025 and 2025/2026; exactly one may be current               |
| core.semester                        | 4    | Two per year; exactly one may be current                          |
| people.person                        | 44   | Every human: 36 students, 7 lecturers, 1 external TA              |
| people.student                       | 36   | 32 Level-200, 2 Level-400, 2 MPhil                                |
| people.next_of_kin                   | 10   | Emergency contacts; at most one primary per student               |
| people.lecturer                      | 7    | Teaching staff across the three departments                       |
| people.teaching_assistant            | 5    | 2 graduate, 2 undergraduate, 1 external                           |
| academics.course                     | 10   | The catalogue, including two Level-100 prerequisites              |
| academics.course_prerequisite        | 5    | Self-referencing many-to-many                                     |
| academics.course_offering            | 8    | Courses running in First Semester 2025/2026                       |
| academics.enrollment                 | 226  | <b>Functionality 3</b> — 32 students × 7 courses, plus exceptions |
| academics.lecturer_course_assignment | 9    | <b>Functionality 4</b> — 8 leads + 1 co-lecturer                  |
| academics.lecturer_ta_assignment     | 6    | <b>Functionality 5</b>                                            |
| finance.fee_structure                | 5    | Price lists per programme/level/residency                         |
| finance.fee_item                     | 26   | Tuition, facility fee, examination, SRC dues, ...                 |
| finance.student_bill                 | 36   | <b>Functionality 2</b> — one per student per year                 |
| finance.bill_line                    | 190  | Frozen copy of the charges on each bill                           |
| finance.payment                      | 54   | Confirmed, pending and reversed payments                          |
| app.app_user                         | 18   | Logins; passwords bcrypt-hashed                                   |
| app.user_session                     | –    | Populated at runtime by the web application                       |
| app.audit_log                        | 18   | Append-only trail of security-relevant actions                    |

## 2.4 Keys, constraints and data integrity

Every table uses a surrogate primary key generated with `GENERATED ALWAYS AS IDENTITY`, and carries a natural unique key alongside it (`student_number`, `staff_number`, `course_code`, `receipt_number`). Surrogate keys keep foreign keys narrow and stable; the natural key is what humans quote and what the seed scripts join on, so the scripts contain no hard-coded ids.

The database enforces **36 foreign keys** and **37 CHECK constraints**. The rules that are genuinely interesting are the ones that stop the data from telling a story that cannot be true:

| Constraint                   | What it prevents                                                                |
|------------------------------|---------------------------------------------------------------------------------|
| student_hall_ck              | A resident student without a hall, or a commuter assigned to one                |
| ta_student_link_ck           | A graduate/undergraduate TA with no student record, or an external TA with one  |
| enrollment_dropped_ck        | A registration marked <i>dropped</i> with no date, or a date without the status |
| enrollment_uq                | Registering for the same offering twice                                         |
| student_bill_uq              | Two bills for the same student in one academic year                             |
| student_bill_due_ck          | A due date that falls before the issue date                                     |
| payment_amount_ck            | A zero or negative payment                                                      |
| lecturer_course_one_lead_uq  | Two lead lecturers on one offering <b>partial index</b>                         |
| academic_year_one_current_uq | Two academic years flagged current <b>partial index</b>                         |
| next_of_kin_one_primary_uq   | Two primary emergency contacts for one student <b>partial index</b>             |

#### A NOTE ON PARTIAL UNIQUE INDEXES

"At most one lead lecturer per offering" cannot be written as a `UNIQUE` constraint, because *several* co-lecturers are allowed on the same offering. PostgreSQL's partial index expresses it exactly: `CREATE UNIQUE INDEX ... ON lecturer_course_assignment (offering_id) WHERE teaching_role = 'lead_lecturer' AND is_active`. The same technique enforces the single current academic year and the single primary next-of-kin.

The same trick solves a subtler problem in `lecturer_ta_assignment`. Its `offering_id` is nullable, because a TA may support a lecturer generally rather than on one named course. A plain `UNIQUE(lecturer_id, ta_id, offering_id)` would not stop duplicates, since SQL treats each `NULL` as distinct. Two partial indexes — one for rows with an offering, one for rows without — close that gap.

## 2.5 Indexing strategy

PostgreSQL already creates an index for every primary key and unique constraint, so only foreign-key columns and frequent filter or sort columns are indexed explicitly. Indexing everything would slow down writes for no benefit. Of the 97 indexes present, the ones that matter most carry the outstanding-fees calculation:

```
-- The fees function only ever sums CONFIRMED money, so the index
-- stores only those rows: smaller, and it answers the query directly.
CREATE INDEX payment_confirmed_ix ON finance.payment (bill_id, amount)
    WHERE status = 'confirmed';

-- Live sessions only - expired rows are never looked up.
CREATE INDEX session_live_ix ON app.user_session (expires_at)
    WHERE revoked_at IS NULL;
```

## 2.6 Triggers and maintained values

Four trigger functions keep derived data honest and enforce rules that a `CHECK` constraint cannot express, because they must look at other rows:

| Trigger function                              | Behaviour                                                                                               |
|-----------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <code>core.fn_set_updated_at</code>           | Stamps <code>updated_at</code> on every update, across all 18 tables that carry the column              |
| <code>finance.fn_refresh_bill_total</code>    | Keeps <code>student_bill.total_amount</code> equal to the sum of its bill lines                         |
| <code>finance.fn_refresh_bill_status</code>   | Recalculates <i>issued</i> / <i>part_paid</i> / <i>paid</i> / <i>overdue</i> whenever a payment changes |
| <code>academics.fn_validate_enrollment</code> | Rejects registration when the student is not active, the class is full, or registration is closed       |

`student_bill.total_amount` is a deliberate, maintained denormalisation. The outstanding-fees report reads it for every student on every call; recomputing `SUM(bill_line.amount)` each time would be wasteful. The trigger guarantees it can never drift from the sum of its lines, which is the only condition under which such a denormalisation is safe.



### 3. The Required Function — Outstanding Fees as a JSON Array

#### THE REQUIREMENT, VERBATIM

"Create a database function that will calculate the outstanding fees for each student in your database and return the output in json array."

#### 3.1 Specification

```
finance.fn_outstanding_fees_json(  
    p_academic_year_id INTEGER DEFAULT NULL,    -- NULL = every academic year  
    p_programme_id     INTEGER DEFAULT NULL,    -- NULL = every programme  
    p_only_indebted     BOOLEAN DEFAULT FALSE   -- TRUE = hide fully paid students  
) RETURNS json
```

All three parameters are optional, so the no-argument call `SELECT finance.fn_outstanding_fees_json();` satisfies the brief exactly while the same function still serves the filtered views the dashboard and the REST API need. The definition of the figure it returns is:

$$\text{outstanding} = \Sigma (\text{amount billed}) - \Sigma (\text{payments with status confirmed})$$

Payments that are `pending`, `reversed` or `failed` are deliberately excluded. A cheque lodged but not yet cleared has not reduced what the student owes, and a reversed mobile-money transfer never did. This is the single most important business rule in the fees module, so the sample data contains one pending payment and one reversed payment purely so the exclusion can be demonstrated (§8).

#### 3.2 Implementation

The function is written in three steps as a chain of CTEs. Abridged, with the reasoning in comments:

```

WITH bill_payment AS (
    -- Step 1: collapse each bill to (billed, confirmed paid, last payment).
    -- The FILTER clause is what implements the confirmed-only rule.
    SELECT b.bill_id, b.student_id, b.total_amount, b.due_date, b.status,
           COALESCE(SUM(p.amount) FILTER (WHERE p.status = 'confirmed'), 0) AS paid,
           MAX(p.payment_date) FILTER (WHERE p.status = 'confirmed') AS last_payment_date
    FROM   finance.student_bill b
    LEFT JOIN finance.payment p ON p.bill_id = b.bill_id
    WHERE  b.status <> 'cancelled'
           AND (p_academic_year_id IS NULL OR b.academic_year_id = p_academic_year_id)
    GROUP BY b.bill_id
),
student_totals AS (
    -- Step 2: roll the bills up to the student and nest the per-bill
    -- breakdown. The LEFT JOIN keeps students who have never been billed,
    -- so "each student in your database" really does mean each student.
    SELECT s.student_id, s.student_number, pr.first_name, pr.last_name, ...
           COALESCE(SUM(bp.total_amount), 0) AS total_billed,
           COALESCE(SUM(bp.paid), 0) AS total_paid,
           COALESCE(SUM(bp.total_amount), 0)
             - COALESCE(SUM(bp.paid), 0) AS outstanding_balance,
           COALESCE(json_agg(json_build_object(...))
                    FILTER (WHERE bp.bill_id IS NOT NULL), '[]'::json) AS bills
    FROM   people.student s
    JOIN   people.person pr ON pr.person_id = s.person_id
    JOIN   core.programme pg ON pg.programme_id = s.programme_id
    LEFT JOIN bill_payment bp ON bp.student_id = s.student_id
    WHERE  (p_programme_id IS NULL OR s.programme_id = p_programme_id)
    GROUP BY s.student_id, pr.person_id, pg.programme_id
)
-- Step 3: shape the array. COALESCE guarantees a valid empty array
-- rather than SQL NULL when no student matches the filters.
SELECT COALESCE(json_agg(json_build_object(
    'student_id', t.student_id, 'student_number', t.student_number,
    'full_name', ..., 'total_billed', ROUND(t.total_billed, 2),
    'total_paid', ROUND(t.total_paid, 2),
    'outstanding_balance', ROUND(t.outstanding_balance, 2),
    'payment_status', CASE WHEN t.total_billed = 0 THEN 'NOT BILLED'
                          WHEN t.outstanding_balance <= 0 THEN 'FULLY PAID'
                          WHEN t.total_paid = 0 THEN 'NO PAYMENT'
                          ELSE 'PART PAID' END,
    'bills', t.bills)
    ORDER BY t.outstanding_balance DESC, t.student_number), '[]'::json)
INTO v_result
FROM student_totals t
WHERE (NOT p_only_indebted OR t.outstanding_balance > 0);

```

#### Four details that are easy to get wrong

- **LEFT JOIN from student, not student\_bill**. An inner join would silently drop any student who has never been billed. The brief says *each student*, so an unbilled student must still appear — with a balance of zero and the status `NOT BILLED`.
- **FILTER rather than a WHERE clause**. Filtering the join to confirmed payments in `WHERE` would also remove bills that have *only* unconfirmed payments, hiding exactly the debt the report exists to surface.
- **COALESCE(json\_agg(...), '[]')**. With no matching rows, `json_agg` returns `SQL NULL`, not an empty array. A client doing `data.length` would then fail. The `COALESCE` guarantees the return value is always a JSON array — which is what the brief asks for.
- **json\_agg(...) FILTER (WHERE bp.bill\_id IS NOT NULL)**. Without the filter, an unbilled student's nested `bills` array would contain one object full of nulls instead of being empty.

### 3.3 Worked example

The author's own record, as returned by the function:

```
SELECT jsonb_pretty(e::jsonb)
FROM   json_array_elements(finance.fn_outstanding_fees_json()) e
WHERE  e->>'student_number' = '22128981';
```

```
{
  "student_id": 36,
  "student_number": "22128981",
  "full_name": "Gideon Elorm Glago",
  "email": "gideon.gлаго@st.ug.edu.gh",
  "programme": "BSc Computer Engineering",
  "programme_code": "BSC-CPEN",
  "level": 200,
  "student_status": "active",
  "residential_status": "non-resident",
  "currency": "GHS",
  "total_billed": 4470.00,
  "total_paid": 3200.00,
  "outstanding_balance": 1270.00,
  "percentage_paid": 71.59,
  "payment_status": "PART PAID",
  "last_payment_date": "2025-11-18",
  "bill_count": 1,
  "bills": [
    {
      "bill_id": 36,
      "bill_reference": "BILL-20252026-00036",
      "academic_year": "2025/2026",
      "amount_billed": 4470.00,
      "amount_paid": 3200.00,
      "balance": 1270.00,
      "due_date": "2025-10-31",
      "bill_status": "overdue",
      "payments_recorded": 2,
      "is_overdue": true
    }
  ]
}
```

The arithmetic is auditable by hand: the Level-200 non-resident fee structure charges GH¢2,050.00 facility fee + GH¢1,890.00 tuition + GH¢320.00 examination + GH¢150.00 SRC dues + GH¢60.00 insurance = **GH¢4,470.00**. Two instalments of GH¢2,000.00 and GH¢1,200.00 were confirmed, leaving **GH¢1,270.00** outstanding — 71.59% paid.

### 3.4 Proof of correctness

Asserting that a function is correct is not the same as showing it. Section 7d of `07_verification.sql` recomputes every balance a second time, independently, in plain SQL that shares no code with the function, and compares the two:

```

WITH from_function AS (
  SELECT (e->>'student_id')::INT, (e->>'outstanding_balance')::NUMERIC
  FROM   json_array_elements(finance.fn_outstanding_fees_json()) AS e
),
from_plain_sql AS (
  SELECT s.student_id,
         COALESCE(SUM(b.total_amount), 0)
         - COALESCE(SUM((SELECT SUM(p.amount) FROM finance.payment p
                           WHERE p.bill_id = b.bill_id AND p.status = 'confirmed')), 0)
  FROM   people.student s
  LEFT JOIN finance.student_bill b ON b.student_id = s.student_id
                                     AND b.status <> 'cancelled'

  GROUP BY s.student_id
)
SELECT COUNT(*) AS students_compared,
       COUNT(*) FILTER (WHERE fn_outstanding <> sql_outstanding) AS mismatches,
       SUM(ABS(fn_outstanding - sql_outstanding)) AS total_difference
FROM   from_function JOIN from_plain_sql USING (student_id);

```

```

students_compared | mismatches | total_difference
-----+-----+-----
36 | 0 | 0.00

```

#### RESULT

36 students compared, **0 mismatches**, total difference **0.00**. The full captured output is in `docs/verification_output.txt`.

The type of the return value is also asserted rather than assumed:

```

json_type | elements_in_array
-----+-----
array | 36

```

## 4. Supporting Stored Functions

The outstanding-fees function is the required deliverable, but a database that exposes only that one function would push the rest of the logic into application code — where it would then have to be written twice, once for the Next.js app and once for the Project 2 API. Nineteen callable functions and four trigger functions were therefore written, so that both clients share one tested surface.

| Function                                                  | Func. | Purpose                                                     |
|-----------------------------------------------------------|-------|-------------------------------------------------------------|
| <code>finance.fn_outstanding_fees_json</code><br>required | 2     | Outstanding fees for every student, as a JSON array         |
| <code>finance.fn_student_outstanding_balance</code>       | 2     | Scalar balance for one student                              |
| <code>finance.fn_student_fee_statement_json</code>        | 2     | Bills, line items and payment history for one student       |
| <code>finance.fn_generate_student_bill</code>             | 2     | Creates a bill by copying the fee structure onto bill lines |
| <code>finance.fn_record_payment</code>                    | 2     | Records a payment, returns the receipt and the new balance  |
| <code>finance.fn_fees_summary_json</code>                 | 2     | Department-wide collection statistics                       |
| <code>people.fn_student_profile_json</code>               | 1     | Complete personal, contact and academic record              |
| <code>people.fn_students_json</code>                      | 1     | Searchable student directory                                |
| <code>academics.fn_enroll_student</code>                  | 3     | Registers a student for a course offering                   |
| <code>academics.fn_drop_enrollment</code>                 | 3     | Withdraws a student, preserving the history                 |
| <code>academics.fn_student_enrollments_json</code>        | 3     | A student's courses with lecturer and timetable             |
| <code>academics.fn_class_list_json</code>                 | 3     | The class register for one offering                         |
| <code>academics.fn_assign_lecturer_to_course</code>       | 4     | Assigns a lecturer to an offering in a role                 |
| <code>academics.fn_lecturer_workload_json</code>          | 4, 5  | A lecturer's courses, hours and TAs                         |
| <code>academics.fn_assign_ta_to_lecturer</code>           | 5     | Assigns a TA, guarding the weekly hour cap                  |
| <code>academics.fn_ta_assignments_json</code>             | 5     | Every lecturer-to-TA assignment                             |
| <code>app.fn_register_user</code>                         | —     | Creates a login and writes the audit row                    |
| <code>app.fn_user_context_json</code>                     | —     | Resolves a user across all role tables                      |
| <code>app.fn_dashboard_stats_json</code>                  | —     | Headline counters for the dashboard                         |

Two of these encode rules that would be easy to lose if they lived in application code. `fn_assign_ta_to_lecturer` refuses an assignment that would push a teaching assistant past their contracted weekly hours, and refuses to attach a TA to a course the named lecturer does not actually teach. `fn_enroll_student` relies on the enrolment trigger to reject a full class or an inactive student. Because these live in the database, they hold whether the caller is `psql`, the web application, or the REST API.

## 5. Sample Data — the CPEN 208 Class

---

### 5.1 What was loaded

The brief says to use the class as the data sample. `06_seed_data.sql` models the CPEN 208 class of First Semester 2025/2026 and everything around it:

- **32 Level-200 students** reading BSc Computer Engineering, each registered for all seven Level-200 courses;
- **2 Level-400 students** and **2 MPhil students** who also serve as teaching assistants, which is what exercises the person-supertype design;
- **7 lecturers** across Computer Engineering, Mathematics and the UGRG office;
- **5 teaching assistants** — two graduate, two undergraduate, one external national-service appointee;
- **10 courses, 8 semester offerings, 226 registrations;**
- **36 bills** totalling **GHC191,240.00**, against which **GHC115,093.50** has been collected.

The data is deliberately imperfect, because uniformly perfect data tests nothing. Two students dropped MATH 223 after the add/drop window; one is repeating CPEN 201; one payment is a cheque still awaiting clearance; one mobile-money transfer was reversed by the provider; one student's balance was settled outright by a GETFund scholarship. The instalment plan each student follows is derived deterministically from their id, so re-running the script always produces byte-identical data.

|                                   |                                |                                 |                                    |                                |
|-----------------------------------|--------------------------------|---------------------------------|------------------------------------|--------------------------------|
| <b>GHC191,240</b><br>TOTAL BILLED | <b>GHC115,094</b><br>COLLECTED | <b>GHC76,147</b><br>OUTSTANDING | <b>60.2%</b><br>COLLECTION<br>RATE | <b>15 / 21</b><br>PAID / OWING |
|-----------------------------------|--------------------------------|---------------------------------|------------------------------------|--------------------------------|

### 5.2 Privacy assumption

#### STATED EXPLICITLY

Apart from the author's own record (22128981), **all** personal details in the sample — names, dates of birth, Ghana Card numbers, phone numbers, e-mail addresses and fee balances — are synthetic stand-ins for real classmates and staff. They are realistic in *shape*, so that every constraint, trigger and function is genuinely exercised, but they identify nobody. Publishing real classmates' dates of birth, national identity numbers and outstanding fee balances to a public repository would not be defensible, and the brief's instruction to "make all necessary assumptions" is taken to cover this. The lecturers' names are likewise invented and are not intended to correspond to any member of staff.

## 6. The Next.js 14 Application

---

### 6.1 Architecture

Question 2 asks for a Next.js 14 application with login, register and a dashboard. It is built on the App Router with TypeScript in strict mode and Tailwind CSS, and it talks to PostgreSQL directly through a pooled `pg`

client. No ORM is used — the queries already exist in the database as functions, and an ORM would add a translation layer over SQL that is deliberately hand-written.

| Concern     | Choice                                     | Reason                                                                           |
|-------------|--------------------------------------------|----------------------------------------------------------------------------------|
| Framework   | Next.js 14.2, App Router                   | Required by the brief; server components keep the pool and bcrypt off the client |
| Data access | pg pool, cached on <code>globalThis</code> | Hot reload would otherwise leak a pool per edit                                  |
| Mutations   | Server Actions                             | Form posts without hand-written API routes                                       |
| Passwords   | bcrypt, cost 10                            | The plain password never reaches the database or a log                           |
| Sessions    | Opaque token; SHA-256 hash stored          | Revocable, unlike a stateless JWT; a leaked dump cannot be replayed              |
| Validation  | zod                                        | One schema validates in the browser and again on the server                      |

## 6.2 Register, login and sessions

**Registration** validates the form with zod, hashes the password with bcrypt, and calls `app.fn_register_user(...)`. If the user supplies an index number, the new login is linked to the existing `people.person` row, so their real fees and courses appear the moment they sign in. This was verified live: registering the account `efua.lamprey` against index number 22129055 produced a session whose dashboard immediately showed that student's true outstanding balance of GHC1,788.00 and her seven registered courses.

**Login** accepts a username, an index number or an e-mail address. An unknown user and a wrong password return the *identical* message, so the form cannot be used to discover which accounts exist; the code even runs a dummy bcrypt comparison in the unknown-user branch so the two paths take comparable time. Five consecutive failures lock the account for fifteen minutes.

**Sessions** are the part most often done badly. The browser receives a 32-byte random token in an `httpOnly` cookie; the database stores only its SHA-256 hash in `app.user_session`. Anyone who obtains a database dump therefore cannot replay a live session. Sessions expire after seven days and can be revoked server-side, which a stateless JWT could not offer.

### WHY MIDDLEWARE ALONE IS NOT THE SECURITY BOUNDARY

`src/middleware.ts` runs on the Edge runtime, which cannot open a TCP connection to PostgreSQL. It can therefore only check that a session cookie is *present*. The authoritative check is `getCurrentUser()` in the dashboard layout, which validates the token against `app.user_session` on every request. A forged cookie passes middleware and is then rejected by the layout, which redirects to `/login`. Treating middleware as the security boundary — a common mistake — would leave the dashboard open to anyone who set a cookie by hand.

## 6.3 The dashboard

The dashboard is role-aware: a student, a lecturer and an administrator see different navigation and different pages from the same route tree.

| Route                       | Access  | Shows                                                                                              |
|-----------------------------|---------|----------------------------------------------------------------------------------------------------|
| /dashboard                  | any     | Balance, registered courses and level — or, for staff, department-wide counters and fee collection |
| /dashboard/profile          | student | <b>Functionality 1</b> — personal, contact, academic and next-of-kin details                       |
| /dashboard/fees             | student | <b>Functionality 2</b> — bills, itemised charges, payment history                                  |
| /dashboard/courses          | student | <b>Functionality 3</b> — registered courses with lecturer and timetable                            |
| /dashboard/catalogue        | any     | Course offerings with live seat availability                                                       |
| /dashboard/students         | staff   | <b>Functionality 1</b> — the student register                                                      |
| /dashboard/teaching         | staff   | <b>Functionalities 4 &amp; 5</b> — teaching load and TA allocation                                 |
| /dashboard/outstanding-fees | staff   | <b>The required function</b> , rendered as a report                                                |

Every one of these pages reads through `src/lib/queries.ts`, which contains one wrapper per stored function. No fee arithmetic, capacity check or hour cap is reimplemented in TypeScript — the numbers on screen are the numbers the database computed.



## 7. Assumptions Made

The brief instructs: *"Make all necessary assumption on the database name, schemas, tables and attributes."* Those made are recorded here rather than left implicit.

| Area                        | Assumption and justification                                                                                                                                                                                                                               |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Database name               | <code>cpen208_ceds</code> — course code plus "Computer Engineering Department System".                                                                                                                                                                     |
| Currency                    | Ghana Cedis (GHS), stored as <code>NUMERIC(12,2)</code> . Never floating point: $0.1 + 0.2 \neq 0.3$ in binary floating point, and fee balances must be exact.                                                                                             |
| Fee amounts                 | Illustrative but of a realistic order of magnitude for a publicly funded Ghanaian university. Real published figures were not available.                                                                                                                   |
| Billing period              | One bill per student per <i>academic year</i> , not per semester, matching current University of Ghana practice.                                                                                                                                           |
| What counts as paid         | Only <code>confirmed</code> payments reduce the balance. This is the central rule of §3.                                                                                                                                                                   |
| Course codes                | CPEN 201/203/205/207/208, MATH 223 and UGRC 210 are a plausible Level-200 first-semester load. CPEN 208 is the one course named in the brief.                                                                                                              |
| Class size                  | 32 students, a realistic Level-200 Computer Engineering cohort.                                                                                                                                                                                            |
| "Lectures to TA assignment" | Read as <i>lecturers</i> to TA assignment. Modelled as $TA \rightarrow \text{lecturer}$ , optionally scoped to one course offering, since in practice the help is attached to a named course.                                                              |
| TA identity                 | A TA may also be a registered student. Handled by the person supertype rather than by duplicating personal data.                                                                                                                                           |
| Bill dates                  | A bill's <code>issued_on</code> defaults to the first day of the academic year it charges for, not the day the script runs. Without this the scripts would fail when rebuilt after the due date — which is exactly what happened during development (§11). |
| Personal data               | Synthetic apart from the author's own record. See §5.2.                                                                                                                                                                                                    |
| Passwords                   | Every seeded account uses <code>Password123!</code> , hashed with <code>bcrypt</code> via <code>pgcrypto's crypt()</code> . Demonstration only; a real deployment would force a change on first login.                                                     |

## 8. Testing and Verification

`07_verification.sql` produces the evidence that each requirement is met. Running it against the delivered database gives the results below; the complete 366-line output is in `docs/verification_output.txt`.

| §  | Check                                  | Result                                                          |
|----|----------------------------------------|-----------------------------------------------------------------|
| 1  | Schemas created                        | 5 schemas · 23 tables · 6 views · 23 functions                  |
| 2  | Functionality 1 — personal information | 36 students; full JSON profile returned                         |
| 3  | Functionality 3 — enrolment            | 8 offerings; 32 enrolled per Level-200 course; 2 drops recorded |
| 4  | Functionality 4 — lecturer assignment  | 9 assignments; CPEN 208 has 1 lead + 1 co-lecturer              |
| 5  | Functionality 5 — TA assignment        | 6 assignments; no TA over their contracted hours                |
| 6  | Functionality 2 — fees                 | 36 bills; billed, paid and outstanding per student              |
| 7a | Return type is a JSON array            | array , 36 elements <span>pass</span>                           |
| 7d | Cross-check against independent SQL    | 36 compared · 0 mismatches · difference 0.00 <span>pass</span>  |
| 7e | Filtered call — indebted only          | 21 of 36 students <span>pass</span>                             |
| 7f | Pending / reversed payments excluded   | Confirmed below <span>pass</span>                               |

Section 7f is worth showing in full, because it demonstrates the rule that matters most:

```

student_number | receipt_number | amount | status | outstanding_balance
-----+-----+-----+-----+-----
22129027      | RCPT-2025-900001 | 1500.00 | pending | 1788.00
22129048      | RCPT-2025-900002 | 800.00 | reversed | 0.00

```

Student 22129027 has a GHC1,500.00 cheque lodged but not cleared; their balance is unchanged by it. Student 22129048 had a GHC800.00 transfer reversed; their balance likewise ignores it. Had either been counted, both figures would be wrong — and no constraint would have caught it. This is precisely why the check is executed rather than merely asserted.

Beyond SQL, the Next.js application was exercised end to end in a browser: signing in as 22128981 , reading the profile, fees and courses pages, registering a new account linked to index 22129055, and signing in as admin to view the outstanding-fees report. Each step was confirmed against the database — the REGISTER and LOGIN rows in app.audit\_log , the 60-character \$2b\$10\$ bcrypt hash, and the hashed session token with its seven-day expiry.

## 9. Backup and Restore

Four pg\_dump outputs are provided in database/backup/ , so the marker can restore whichever suits:

| File                         | Size   | Restore with                                               |
|------------------------------|--------|------------------------------------------------------------|
| cpen208_ceds_backup.sql      | 244 KB | psql -d newdb -f cpen208_ceds_backup.sql                   |
| cpen208_ceds_backup.dump     | 200 KB | pg_restore -d newdb --no-owner<br>cpen208_ceds_backup.dump |
| cpen208_ceds_schema_only.sql | 135 KB | Structure without data                                     |
| cpen208_ceds_data_only.sql   | 92 KB  | Data without structure                                     |

The plain-SQL backup was restored into a scratch database as part of preparing this submission, and the required function was re-run there: it returned all 36 students correctly. A backup that has never been restored is only a hope, so the restore was tested rather than assumed.

## 10. How to Run Everything

---

```
# 1 – build the database (drops and recreates cpen208_ceds)
cd project-1/database && ./run_all.sh

# 2 – the required function, on its own
psql -d cpen208_ceds -c "SELECT finance.fn_outstanding_fees_json();"

# 3 – the full verification report
psql -d cpen208_ceds -f 07_verification.sql

# 4 – the Next.js 14 application
cd ../web
cp .env.example .env.local          # then set DATABASE_URL
npm install && npm run dev           # http://localhost:3000
```

Demonstration accounts, all with the password `Password123!` : `22128981` (student), `kadanquah` (lecturer), `admin` (administrator).

## 11. Challenges, Limitations and Future Work

---

### Challenges met during development

- **Bills dated relative to "today".** The first run of the seed script failed on `student_bill_due_ck`: `issued_on` defaulted to the current date, which fell *after* the October 2025 due date being loaded. Rather than move the due date, the fix was conceptual — a bill belongs to the academic year it charges for, so `fn_generate_student_bill` now defaults `issued_on` to the first day of that year. The scripts are now reproducible however long after the session they are run.
- **A negative balance.** After adding an explicit instalment plan for the author's record, the dashboard showed a balance of *minus* GHC1,412.00. The explicit payments were being added on top of the pattern-generated ones rather than replacing them. The seed now excludes that student from the generated pattern, and the verification suite gained a check that no student's balance is ever negative.
- **CREATE OR REPLACE does not replace a signature.** Adding a fourth parameter to `fn_generate_student_bill` created a second *overload* instead of replacing the function, and the next call failed as ambiguous. The script now drops the old signature explicitly, so it remains re-runnable against a database built by an earlier version.
- **A 'use server' file may export only async functions.** Exporting the shared form-state object from `actions.ts` compiled cleanly but crashed at runtime on the first login. The type and constant moved to `lib/formState.ts`.

### Limitations

- Grades are stored but there is no transcript or CGPA recalculation routine; `student.cgpa` is set from the sample data rather than derived.
- Prerequisites are recorded but not enforced at registration; the trigger checks capacity, status and the registration window only.
- There is no timetable-clash detection — a student may register for two courses meeting at the same hour.
- Fee payment is recorded, not *taken*: there is no payment-gateway integration.
- Row-level security is not enabled; access control lives in the application and API layers.

### Future work

- Enforce prerequisites and timetable clashes inside `fn_enroll_student`, where every client would inherit them.
- Add PostgreSQL row-level security so the database itself restricts a student to their own rows.
- Partition `finance.payment` and `app.audit_log` by year once they grow.
- Materialise the outstanding-fees report if the student population reaches a scale where per-request computation is measurable.

## 12. Conclusion

---

Every item in the Project 1 brief has been delivered: a PostgreSQL database named `cpen208_ceds` with five purposeful schemas, 23 tables covering all five required functionalities, insert scripts that populate every one of them with the CPEN 208 class, and — the headline requirement —

`finance.fn_outstanding_fees_json()` , which calculates the outstanding fees for each student and returns them as a JSON array. Alongside it, a Next.js 14 application provides login, registration and a role-aware dashboard, and four restore-tested backups accompany the scripts.

The decision that shaped everything else was to put the business rules in the database rather than in application code. "Outstanding fees" is defined once, in SQL, and is therefore the same number whether it is read from `psql` , rendered on the dashboard, or — as Project 2 goes on to show — returned over HTTP. Correctness was not asserted but demonstrated: the function's output is cross-checked against independently written SQL for all 36 students, with zero mismatches, and the exclusion of pending and reversed payments is proved against data planted specifically to test it.